Chapitre 18 — Python avancé & Algorithmique

PARTIE 1 — LE COURS COMPLET

1. Récursivité

Une **fonction récursive** est une fonction qui s'appelle elle-même. Elle doit impérativement comporter un **cas de base** (condition d'arrêt, sans appel récursif) et un **cas récursif** (qui rapproche du cas de base à chaque appel).

```
def factorielle(n):
    if n == 0: # cas de base
        return 1
    else: # cas recursif
        return n * factorielle(n - 1)
```

Attention : sans cas de base correctement atteint, la récursion ne s'arrête jamais et Python lève une erreur RecursionError (pile d'appels trop profonde).

2. Matrices 2D (listes de listes)

En Python, une matrice se représente naturellement comme une **liste de listes** : chaque sous-liste est une ligne de la matrice.

```
M = [[1, 2, 3],
     [4, 5, 6]]
print(M[0][2]) # affiche 3 (ligne 0, colonne 2)
print(len(M)) # affiche 2 (nombre de lignes)
print(len(M[0])) # affiche 3 (nombre de colonnes)

def transposee(M):
    lignes, colonnes = len(M), len(M[0])
    return [[M[i][j] for i in range(lignes)] for j in range(colonnes)]
```

3. Visualisation avec matplotlib

Le module matplotlib.pyplot permet de tracer des courbes, nuages de points ou histogrammes directement à partir de listes de valeurs.

```
import matplotlib.pyplot as plt

x = [i for i in range(11)]
y = [i**2 for i in x]
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Courbe de x -> x**2")
plt.show()
```

Chapitre 18 — Python avancé & Algorithmique

4. Simulation d'une loi binomiale

On peut simuler un schéma de Bernoulli (voir chapitre sur la loi binomiale) en tirant des nombres aléatoires avec le module random, puis compter les succès pour obtenir une réalisation d'une variable suivant B(n,p).

```
import random

def simulation_binomiale(n, p, nb_essais):
    resultats = []
    for _ in range(nb_essais):
        succes = 0
        for _ in range(n):
            if random.random() < p: # succes avec probabilite p
                succes += 1
        resultats.append(succes)
    return resultats
```

5. Simulation d'une loi normale

Le module random propose directement une fonction pour simuler une loi normale, sans avoir à la reconstruire à partir d'une loi uniforme.

```
import random

# Simule 1000 valeurs d'une loi normale N(mu=10, sigma=2)
valeurs = [random.gauss(10, 2) for _ in range(1000)]
moyenne = sum(valeurs) / len(valeurs)
print(moyenne) # proche de 10
```

6. Méthode de Newton (recherche de racine, optionnel)

La méthode de Newton permet d'approcher numériquement une solution de l'équation f(x)=0, en partant d'une valeur x0 et en itérant $x_{suivant} = x - f(x)/f'(x)$.

```
def newton(f, fprime, x0, epsilon=1e-6, n_max=100):
    x = x0
    for _ in range(n_max):
        x_suivant = x - f(x) / fprime(x)
        if abs(x_suivant - x) < epsilon:
            return x_suivant
        x = x_suivant
    return x

# Exemple : approcher racine(2), solution positive de x**2 - 2 = 0
racine_2 = newton(lambda x: x**2 - 2, lambda x: 2*x, x0=1)
```

Chapitre 18 — Python avancé & Algorithmique

7. Intégrales numériques (méthode des rectangles)

Quand une primitive est difficile ou impossible à exprimer, on peut approcher une intégrale par une somme d'aires de rectangles, en subdivisant [a,b] en n intervalles égaux.

```
def integrale_rectangles(f, a, b, n):
    largeur = (b - a) / n
    aire = 0
    for i in range(n):
        x_i = a + i * largeur
        aire += f(x_i) * largeur
    return aire

# Exemple : approcher l'integrale de x**2 entre 0 et 1 (valeur exacte 1/3)
approx = integrale_rectangles(lambda x: x**2, 0, 1, 1000)
```

Plus n est grand, plus l'approximation est précise (et plus le temps de calcul augmente) : c'est un compromis classique entre précision et coût de calcul.

8. Méthode — construire une simulation Monte-Carlo

1. Identifier l'expérience aléatoire à simuler et la traduire en instructions Python (random.random(), random.gauss(), random.uniform()...).

2. Répéter l'expérience un grand nombre de fois dans une boucle, en stockant ou comptant les résultats.

3. Calculer une fréquence, une moyenne ou une proportion à partir des résultats accumulés.

4. Comparer le résultat simulé à la valeur théorique attendue, et vérifier que l'approximation s'améliore quand le nombre de répétitions augmente.

9. Exemple résolu — estimation de π par Monte-Carlo

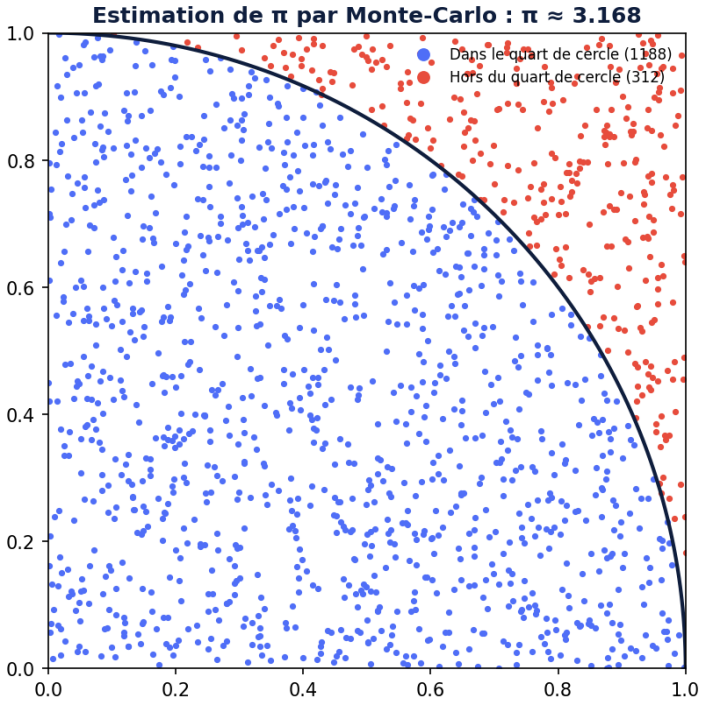
Exemple

On tire au hasard des points (x,y) avec x et y uniformes dans [0,1]. La probabilité qu'un point tombe dans le quart de disque de rayon 1 ($x^2+y^2 \leq 1$) vaut $\pi/4$ (rapport des aires). En estimant cette probabilité par simulation, on en déduit une estimation de π.

```
import random

def estimation_pi(nb_points):
    dans_le_cercle = 0
    for _ in range(nb_points):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        if x**2 + y**2 <= 1:
            dans_le_cercle += 1
    return 4 * dans_le_cercle / nb_points

print(estimation_pi(1500)) # resultat proche de 3.14, variable a chaque execution
```



10. Erreurs fréquentes à éviter

Attention — Écrire une fonction récursive sans cas de base (ou un cas de base jamais atteint), provoquant une erreur RecursionError.

Attention — Confondre les indices ligne/colonne d'une matrice : M[i][j] désigne la ligne i, colonne j — pas l'inverse.

Attention — Utiliser random.random() (qui renvoie un flottant entre 0 et 1) là où l'on voulait random.gauss() (loi normale) ou l'inverse.

Attention — Prendre un n trop petit dans une méthode des rectangles ou une simulation Monte-Carlo, donnant un résultat peu précis et instable d'une exécution à l'autre.

Attention — Oublier que f'(x) (dérivée) doit être fournie séparément dans la méthode de Newton — Python ne dérive pas automatiquement une fonction.

Chapitre 18 — Python avancé & Algorithmique

À retenir absolument

Récursivité : cas de base + cas récursif · Matrice = liste de listes, M[i][j] = ligne i, colonne j · matplotlib : plt.plot(x,y) puis plt.show() · Simulation binomiale : compter les succès de random.random()<p répétés n fois · Loi normale : random.gauss(mu,sigma) · Newton : x_suivant = x - f(x)/f'(x) · Intégrale (rectangles) : somme de f(x_i)×largeur.

Chapitre 18 — Python avancé & Algorithmique

PARTIE 2 — QUIZ D'ÉVALUATION (10 QUESTIONS)

Q1 (QCM) — Une fonction récursive doit toujours comporter : a) une boucle for b) un cas de base c) un import

Correction : Réponse b).

Q2 (Application) — Que renvoie factorielle(3) avec la fonction du cours ?

Correction : factorielle(3) = 3×factorielle(2) = 3×2×factorielle(1) = 3×2×1×factorielle(0) = 3×2×1×1 = 6.

Q3 (Application) — Pour M=[[1,2],[3,4]], que vaut M[1][0] ?

Correction : M[1][0] = 3 (ligne d'indice 1, colonne d'indice 0).

Q4 (Vrai/Faux) — random.random() renvoie toujours un nombre entier.

Correction : FAUX — random.random() renvoie un nombre flottant (décimal) entre 0 et 1 (0 inclus, 1 exclu).

Q5 (QCM) — Pour simuler une loi normale N(5,2) en Python, on utilise : a) random.random() b) random.gauss(5,2) c) random.randint(5,2)

Correction : Réponse b).

Q6 (Application) — Dans la méthode des rectangles avec a=0, b=2, n=4, quelle est la largeur de chaque rectangle ?

Correction : largeur = (b-a)/n = (2-0)/4 = 0,5.

Q7 (Vrai/Faux) — Plus n est grand dans une méthode des rectangles, plus l'approximation de l'intégrale est précise.

Correction : VRAI — augmenter n rapproche la somme des rectangles de l'aire réelle sous la courbe.

Q8 (QCM) — Dans la méthode de Newton, il faut fournir : a) seulement f b) f et sa dérivée f' c) seulement f'

Correction : Réponse b).

Q9 (Application) — Dans la simulation Monte-Carlo de π, quel facteur multiplie le rapport dans_le_cercle/nb_points ?

Correction : 4 (car le quart de cercle a une aire $\pi/4$, donc $\pi = 4\times$(aire du quart de cercle)).

Q10 (Vrai/Faux) — Une simulation Monte-Carlo donne exactement le même résultat à chaque exécution.

Correction : FAUX — le résultat varie légèrement à chaque exécution car il dépend de tirages aléatoires, mais il converge vers la valeur théorique quand le nombre de répétitions augmente.

Chapitre 18 — Python avancé & Algorithmique

PARTIE 3 — 45 QUESTIONS-RÉPONSES DE RÉVISION

Q1 : Qu'est-ce qu'une fonction récursive ?

RÉPONSE : Une fonction qui s'appelle elle-même dans son propre corps.

Q2 : Quels sont les deux éléments indispensables d'une fonction récursive ?

RÉPONSE : Un cas de base (condition d'arrêt sans appel récursif) et un cas récursif (qui se rapproche du cas de base à chaque appel).

Q3 : Que se passe-t-il si une fonction récursive n'atteint jamais son cas de base ?

RÉPONSE : Python lève une erreur RecursionError, la pile d'appels devenant trop profonde.

Q4 : Comment représente-t-on naturellement une matrice en Python ?

RÉPONSE : Comme une liste de listes, chaque sous-liste représentant une ligne de la matrice.

Q5 : Pour une matrice M, que désigne M[i][j] ?

RÉPONSE : L'élément situé à la ligne d'indice i et à la colonne d'indice j.

Q6 : Comment obtient-on le nombre de lignes d'une matrice M en Python ?

RÉPONSE : Avec len(M).

Q7 : Comment obtient-on le nombre de colonnes d'une matrice M (supposée non vide) en Python ?

RÉPONSE : Avec len(M[0]), la longueur de la première ligne.

Q8 : Quel module Python permet de tracer des courbes et graphiques ?

RÉPONSE : matplotlib.pyplot, généralement importé avec 'import matplotlib.pyplot as plt'.

Q9 : Quelles sont les deux instructions de base pour tracer puis afficher une courbe avec matplotlib ?

RÉPONSE : plt.plot(x, y) pour tracer, puis plt.show() pour afficher la fenêtre graphique.

Q10 : Que renvoie l'instruction random.random() ?

RÉPONSE : Un nombre flottant aléatoire entre 0 (inclus) et 1 (exclu).

Q11 : Comment simuler une épreuve de Bernoulli de paramètre p en Python ?

RÉPONSE : En testant si random.random() < p : la condition est vraie avec probabilité p, ce qui modélise un succès.

Q12 : Comment simuler directement une valeur suivant une loi normale N(mu,sigma) en Python ?

RÉPONSE : Avec random.gauss(mu, sigma).

Q13 : Pourquoi répète-t-on une simulation un grand nombre de fois avant de conclure ?

RÉPONSE : Parce qu'une seule simulation peut être peu représentative ; répéter l'expérience permet d'obtenir une fréquence ou une moyenne stable, proche de la valeur théorique.

Chapitre 18 — Python avancé & Algorithmique

Q14 : Qu'est-ce que la méthode des rectangles en calcul intégral ?

RÉPONSE : Une méthode qui approche l'aire sous une courbe (une intégrale) par la somme des aires de rectangles construits sur une subdivision de l'intervalle.

Q15 : Quelle est la formule de la largeur de chaque rectangle dans la méthode des rectangles, pour une subdivision en n parties de [a,b] ?

RÉPONSE : largeur = (b-a)/n.

Q16 : Comment varie la précision de la méthode des rectangles lorsque n augmente ?

RÉPONSE : Elle s'améliore : plus n est grand, plus l'approximation de l'intégrale se rapproche de la valeur exacte.

Q17 : Quel est l'inconvénient d'augmenter n dans une méthode des rectangles ou une simulation Monte-Carlo ?

RÉPONSE : Le temps de calcul augmente, ce qui peut rendre le programme plus lent à s'exécuter.

Q18 : Qu'est-ce que la méthode de Newton permet d'approcher ?

RÉPONSE : Une solution (racine) d'une équation de la forme f(x)=0, de façon numérique et itérative.

Q19 : Quelle est la formule d'itération de la méthode de Newton ?

RÉPONSE : x_suivant = x - f(x)/f'(x).

Q20 : Pourquoi la méthode de Newton nécessite-t-elle de connaître f' (la dérivée de f) ?

RÉPONSE : Parce que la formule d'itération utilise directement f'(x) pour calculer chaque nouvelle approximation.

Q21 : Quand arrête-t-on les itérations de la méthode de Newton dans l'algorithme du cours ?

RÉPONSE : Quand la différence entre deux approximations successives (|x_suivant - x|) devient inférieure à un seuil de précision epsilon.

Q22 : Pourquoi ajoute-t-on une limite n_max d'itérations dans l'algorithme de Newton ?

RÉPONSE : Pour éviter une boucle infinie si la méthode ne converge pas (ou converge très lentement) vers une solution.

Q23 : Dans l'exemple d'estimation de π, quelle est la probabilité théorique qu'un point tiré dans [0,1]x[0,1] tombe dans le quart de cercle ?

RÉPONSE : π/4, correspondant au rapport de l'aire du quart de disque de rayon 1 sur l'aire du carré unité.

Q24 : Pourquoi multiplie-t-on par 4 le rapport dans_le_cercle/nb_points dans l'estimation de π ?

RÉPONSE : Parce que ce rapport approche π/4 ; le multiplier par 4 donne directement une estimation de π.

Q25 : Quelle condition teste-t-on pour savoir si un point (x,y) est dans le quart de disque unité ?

RÉPONSE : x**2 + y**2 <= 1.

Q26 : Le résultat d'une simulation Monte-Carlo est-il identique à chaque exécution du programme ?

RÉPONSE : Non — il varie légèrement d'une exécution à l'autre car il repose sur des tirages aléatoires, mais il converge vers la valeur théorique quand le nombre de tirages augmente.

Chapitre 18 — Python avancé & Algorithmique

Q27 : Quelle est la première étape de la méthode pour construire une simulation Monte-Carlo ?

RÉPONSE : Identifier l'expérience aléatoire à simuler et la traduire en instructions Python adaptées (random.random, random.gauss, random.uniform...).

Q28 : Quelle est la dernière étape de la méthode pour construire une simulation Monte-Carlo ?

RÉPONSE : Comparer le résultat simulé à la valeur théorique attendue, et vérifier que l'approximation s'améliore avec un plus grand nombre de répétitions.

Q29 : Qu'est-ce qu'une liste en compréhension en Python (utilisée par exemple pour transposer une matrice) ?

RÉPONSE : Une syntaxe compacte permettant de construire une liste directement à partir d'une boucle et d'une condition éventuelle, en une seule ligne.

Q30 : Dans la fonction transposee du cours, que représentent les variables lignes et colonnes ?

RÉPONSE : Respectivement le nombre de lignes et le nombre de colonnes de la matrice M passée en argument.

Q31 : Pourquoi la transposée d'une matrice inverse-t-elle le rôle des lignes et des colonnes ?

RÉPONSE : Parce que l'élément en position (i,j) de la matrice d'origine devient l'élément en position (j,i) dans la matrice transposée.

Q32 : Quel module Python faut-il importer pour utiliser des fonctions aléatoires comme random() ou gauss() ?

RÉPONSE : Le module random (import random).

Q33 : Quelle instruction affiche un graphique après l'avoir construit avec plt.plot() ?

RÉPONSE : plt.show().

Q34 : Comment ajouter un titre à un graphique matplotlib ?

RÉPONSE : Avec l'instruction plt.title("mon titre").

Q35 : Une fonction récursive est-elle toujours plus efficace qu'une boucle classique (itérative) ?

RÉPONSE : Non — la récursivité peut être plus lente et consommer plus de mémoire (pile d'appels) qu'une solution itérative équivalente, mais elle est souvent plus lisible pour certains problèmes.

Q36 : Que se passe-t-il concrètement quand une fonction récursive s'appelle elle-même ?

RÉPONSE : Chaque appel est empilé en mémoire (pile d'appels) en attendant le résultat de l'appel suivant, jusqu'à atteindre le cas de base, puis les résultats sont dépilés et combinés en remontant.

Q37 : Pourquoi teste-t-on x2 + y**2 <= 1 plutôt que x**2 + y**2 < 1 dans l'exemple du cercle ?**

RÉPONSE : Cela ne change rien en pratique pour une simulation continue (la probabilité que le point tombe exactement sur le cercle est nulle), mais l'inégalité large est la convention naturelle pour un disque fermé.

Q38 : Une intégrale calculée par la méthode des rectangles est-elle une valeur exacte ou approchée ?

RÉPONSE : Une valeur approchée — c'est une méthode numérique qui converge vers la valeur exacte mais ne la calcule pas de façon symbolique.

Chapitre 18 — Python avancé & Algorithmique

Q39 : Pourquoi utilise-t-on une méthode numérique (rectangles) plutôt que le calcul direct d'une primitive ?

RÉPONSE : Parce que certaines fonctions n'ont pas de primitive exprimable simplement avec les fonctions usuelles, rendant le calcul exact impossible ou trop complexe.

Q40 : Le paramètre p dans la fonction simulation_binomiale représente-t-il un nombre d'essais ou une probabilité ?

RÉPONSE : Une probabilité — la probabilité de succès à chaque épreuve élémentaire.

Q41 : Le paramètre n dans la fonction simulation_binomiale représente-t-il le nombre d'essais par expérience ou le nombre d'expériences ?

RÉPONSE : Le nombre d'essais (épreuves de Bernoulli) réalisés au cours d'une seule expérience, correspondant au n de la loi B(n,p).

Q42 : Que représente le paramètre nb_essais dans la fonction simulation_binomiale ?

RÉPONSE : Le nombre de fois où l'on répète l'expérience complète (chaque expérience comptant elle-même n épreuves), pour obtenir plusieurs réalisations de la variable aléatoire.

Q43 : Pourquoi le programme d'estimation de π donne-t-il un résultat différent à chaque nouvelle exécution ?

RÉPONSE : Parce qu'il repose sur des tirages aléatoires (random.uniform), qui changent à chaque exécution du programme.

Q44 : Comment améliorer la précision de l'estimation de π obtenue par la méthode de Monte-Carlo ?

RÉPONSE : En augmentant le nombre de points tirés (nb_points), ce qui rapproche la fréquence observée de la probabilité théorique $\pi/4$.

Q45 : Ce chapitre de Python avancé s'appuie-t-il sur des notions déjà vues dans les chapitres précédents de probabilités et d'analyse ?

RÉPONSE : Oui — il applique concrètement par la programmation les notions de loi binomiale, loi normale, intégration et résolution d'équations vues dans les chapitres précédents du programme.